

Objetivo

O objetivo desta atividade prática em laboratório é implementar uma estrutura de dados de **fila** utilizando **vetor circular** e os conceitos de programação orientada a objetos. A fila deverá armazenar valores inteiros e será reutilizada em uma aplicação prática: um **sistema de atendimento por senhas**.

Parte 1 – Implementação da Fila

1. Inicialmente crie os seguintes arquivos fonte:

- **Fila**: interface da fila;
- **FilaVetor**: classe que implementa a fila com vetor circular;
- **FilaMain**: classe com o método principal para demonstrar o funcionamento da fila e da aplicação de atendimento.

2. Defina uma interface **Fila** com as operações básicas da estrutura:

```
public interface Fila {  
    void enqueue(int v);  
    int dequeue();  
    boolean isEmpty();  
    void reset();  
}
```

3. A classe **FilaVetor** deve possuir, no mínimo, os seguintes atributos:

```
private int tam;  
private int n;  
private int ini;  
private int[] vet;
```

4. Significado dos atributos:

- **tam**: capacidade máxima da fila;
- **n**: quantidade atual de elementos armazenados;
- **ini**: índice da posição do próximo elemento a ser removido;
- **vet**: vetor de inteiros usado para armazenar os elementos.

5. A posição do próximo elemento a ser inserido no final da fila não deve ser armazenada em um atributo separado. Sempre que necessário, ela deve ser calculada pela expressão:

```
fim = (ini + n) % tam;
```

6. Métodos a serem implementados na classe **FilaVetor**:

- (a) **public FilaVetor(int tam)**: construtor que instancia uma fila vazia com capacidade máxima igual a **tam**;
- (b) **public void enqueue(int v)**: insere o valor **v** no final da fila. Se a fila estiver cheia, o método deve lançar erro ou interromper a execução com mensagem apropriada;
- (c) **public int dequeue()**: remove e retorna o elemento do início da fila. Se a fila estiver vazia, o método deve lançar erro ou interromper a execução com mensagem apropriada;
- (d) **public boolean isEmpty()**: retorna **true** se a fila estiver vazia, ou **false** caso contrário;
- (e) **public void reset()**: esvazia a fila, reiniciando os atributos **ini** e **n**;
- (f) **public String toString()**: retorna uma representação textual da fila na ordem correta, do início para o fim;
- (g) **public FilaVetor concatena(FilaVetor f2)**: cria e retorna uma nova fila contendo, na mesma ordem, os elementos da fila atual seguidos dos elementos de **f2**. As filas originais devem permanecer inalteradas;
- (h) **public FilaVetor merge(FilaVetor f2)**: cria e retorna uma nova fila contendo os elementos da fila atual e de **f2** intercalados. Se uma das filas terminar antes da outra, os elementos restantes da fila maior devem ser copiados ao final. As filas originais devem permanecer inalteradas.

7. Se desejar, utilize as figuras a seguir como referência visual para as operações sobre a fila circular:

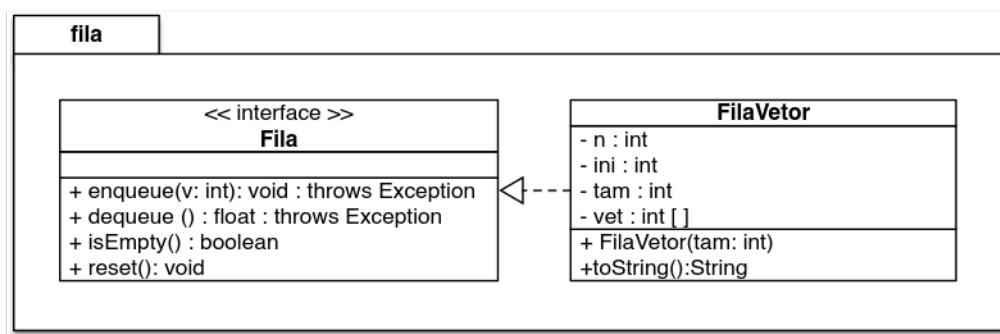


Figura 1: Representação de uma fila com vetor circular

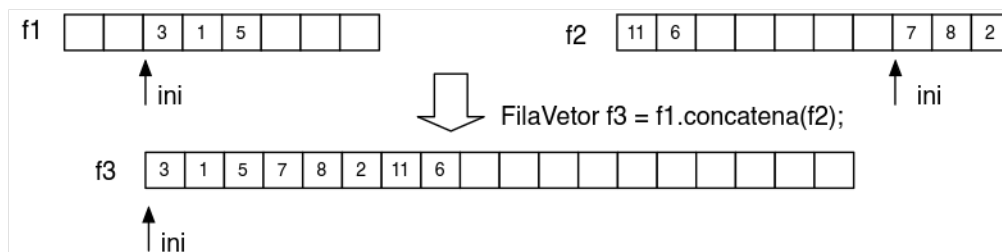


Figura 2: Exemplo da operação concatena

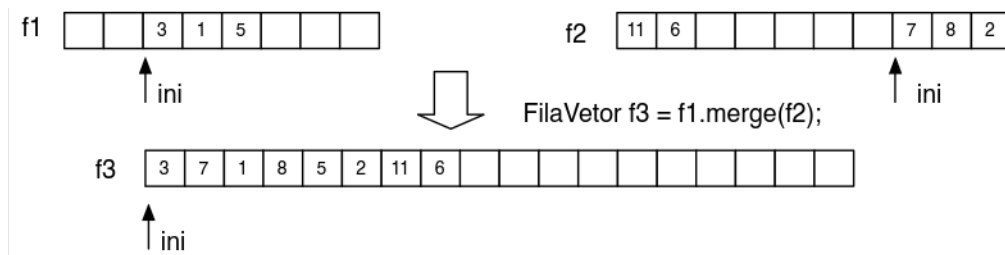


Figura 3: Exemplo da operação merge

Parte 2 – Problema Aplicado: Atendimento por Senhas

Após implementar a fila, utilize-a para representar um **sistema de atendimento por senhas** em um guichê. Cada senha deve ser representada por um valor inteiro. Como a fila é implementada com vetor, a aplicação deve trabalhar com uma **capacidade máxima de atendimento** definida no construtor da fila.

Considere, por exemplo, uma fila criada com capacidade máxima 10. Se todas as posições estiverem ocupadas, novas senhas não poderão ser inseridas até que algum atendimento seja concluído.

Implemente, na classe `FilaMain`, um programa cliente com as seguintes funcionalidades:

1. Criar uma fila de atendimento informando sua capacidade máxima;
2. Inserir novas senhas no final da fila, representando a chegada de novos clientes;
3. Chamar a próxima senha para atendimento, removendo-a do início da fila;
4. Exibir a fila atual de senhas na ordem de atendimento;
5. Reiniciar o atendimento do guichê, esvaziando completamente a fila com a operação **reset**;
6. Demonstrar, no método `main`, um cenário completo com chegadas de clientes, atendimentos, exibição da fila e tentativa de inserção quando a capacidade máxima já tiver sido atingida.

Exemplo de sequência para teste:

```
Capacidade da fila: 5
Chegam as senhas: 101, 102, 103, 104, 105
Atende uma senha
Chega a senha 106
Exibe a fila atual
Reinicia o atendimento
Exibe a fila vazia
```

Observações

- A implementação da fila deve utilizar vetor circular;
- Não crie um atributo extra para armazenar o fim da fila;
- Os métodos `concatena` e `merge` devem produzir novas filas, sem modificar as filas originais;
- Utilize adequadamente os princípios de encapsulamento da programação orientada a objetos;

- Implemente na classe **FilaMain** um exemplo completo demonstrando o funcionamento da fila e da aplicação prática.

Observação Final

Após implementar todas as classes e métodos, desenvolva um programa principal que demonstre claramente o funcionamento completo da fila circular e do sistema de atendimento por senhas.